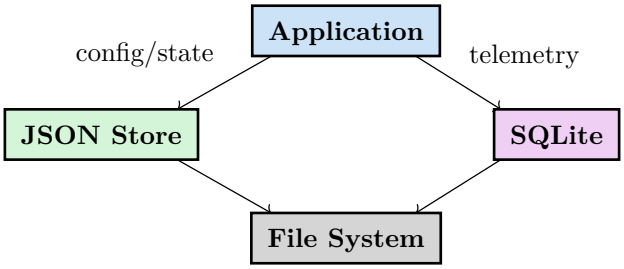
STORAGE LAYER

Persistence Architecture

Storage Overview

WAFT uses a hybrid storage approach combining JSON files for structured data and SQLite for telemetry.



Directory Structure

```
project_root/
├── _realms/                    # Realm data (JSON)
│   └── bureaucracy_realm/
│       ├── manifest.json
│       └── corporations/
│           └── teleport_massive_20250701/
│               ├── manifest.json
│               ├── founders.json
│               ├── departments/
│               ├── employees/
│               └── financial/
├── _work_efforts/              # Johnny Decimal structure
│   ├── 00-09_meta/
│   ├── 10-19_development/
│   └── devlog.md
├── flight_data/                # Telemetry (SQLite)
│   └── flight.db
├── checkpoints/                # Evolution snapshots
│   └── gen_050/
├── seeds/                      # Agent state saves
└── waft.toml                   # Configuration
```

JSON Storage

Beings

```
// _realms/bureaucracy_realm/beings/{being_id}.json
{
  "being_id": "abc-123-def-456",
  "reality_id": "teleport_massive_20250701",
  "state": "learning",
  "skills": {
    "quantum_physics": 8.5,
    "leadership": 7.0
  },
  "memories": [
    {
      "memory_id": "mem_001",
      "content": "Founded the company",
      "memory_type": "achievement",
      "timestamp": "2025-07-01T00:00:00Z"
    }
  ],
  "personality": {
    "visionary": 0.9,
    "determined": 0.85
  },
  "goals": [
    {"goal": "Scale teleportation", "priority": 1.0}
  ],
  "fitness": 0.78,
  "created_at": "2025-07-01T00:00:00Z"
}
```

Corporations

```
// _realms/bureaucracy_realm/corporations/{corp_id}/manifest.json
{
  "corp_id": "teleport_massive_20250701",
  "name": "Teleport Massive",
  "sector": "Quantum Teleportation Technology",
  "mission": "Make distance irrelevant",
  "founded_date": "2025-07-01",
  "capital": "2000000.00",
  "departments": ["Executive", "R&D", "Operations"],
  "employee_count": 5
}
```

SQLite Storage (Flight Recorder)

Schema

```
-- flight_data/flight.db

CREATE TABLE events (
    event_id TEXT PRIMARY KEY,
    timestamp DATETIME NOT NULL,
    event_type TEXT NOT NULL,
    genome_id TEXT,
    parent_id TEXT,
    generation INTEGER,
    fitness REAL,
    metadata JSON,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE genomes (
    genome_id TEXT PRIMARY KEY,
    code_hash TEXT NOT NULL,
    config_hash TEXT NOT NULL,
    prompt_hash TEXT,
    generation INTEGER,
    parent_ids JSON,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE scints (
    scint_id TEXT PRIMARY KEY,
    event_id TEXT REFERENCES events(event_id),
    scint_type TEXT NOT NULL,
    severity REAL NOT NULL,
    evidence TEXT,
    detected_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Indexes for performance
CREATE INDEX idx_events_type ON events(event_type);
CREATE INDEX idx_events_genome ON events(genome_id);
CREATE INDEX idx_events_timestamp ON events(timestamp);
```

Query Examples

```
-- Get fitness history for an agent
SELECT generation, fitness, timestamp
FROM events
WHERE genome_id = 'sha256:abc123'
  AND event_type = 'GYM_EVAL'
ORDER BY generation;

-- Find all SAFETY_VOID scints
SELECT s.*, e.genome_id
FROM scints s
JOIN events e ON s.event_id = e.event_id
WHERE s.scint_type = 'SAFETY_VOID'
ORDER BY s.severity DESC;

-- Get lineage tree
WITH RECURSIVE lineage AS (
  SELECT genome_id, parent_ids, generation, 0 as depth
  FROM genomes WHERE genome_id = ?
```

```
  UNION ALL
  SELECT g.genome_id, g.parent_ids, g.generation, l.depth + 1
  FROM genomes g, lineage l, json_each(l.parent_ids)
  WHERE g.genome_id = json_each.value
)
SELECT * FROM lineage;
```

Configuration Storage

waft.toml

```toml
[project]
name = "my_laboratory"
version = "0.1.0"

[evolution]
population_size = 20
mutation_rate = 0.1
elitism_count = 2
selection_method = "tournament"

[gym]
timeout_seconds = 300
parallel_workers = 4
scenarios = ["syntax", "logic", "safety", "hallucination"]

[storage]
database = "flight_data/flight.db"
checkpoints_dir = "checkpoints"
checkpoint_every = 10

[llm]
provider = "openai"
model = "gpt-4"
temperature = 0.7
```

Storage Metrics

Data Type	Storage	Typical Size
Being	JSON	1-5 KB per Being
Corporation	JSON	10-50 KB per Corporation
Genome	SQLite	500 bytes per genome
Event	SQLite	200 bytes per event
Scint	SQLite	100 bytes per scint
Checkpoint	JSON + Binary	1-10 MB per checkpoint

Backup & Recovery

```bash
# Backup entire laboratory
waft backup --output lab_backup.tar.gz

# Backup just database
sqlite3 flight_data/flight.db ".backup backup.db"

# Restore from backup
waft restore --input lab_backup.tar.gz

# Export to portable format
waft export --format json --output export/
```

STORAGE LAYER | Persistent, Queryable, Recoverable